

The Standalone Demo Bundle

One HTML file that runs the whole S7 console from a `file://` URL
— no server, no Python, no port, no install.

3

API CALLS TO
REPLACE

3.4 MB

AVOIDABLE IF PRE-
RENDERED

~180 KB

TARGET BUNDLE
SIZE

4

PHASES

Answers risk R2 — the sandbox may not serve a port

Plan only. Nothing in this document is built yet.

Prepared 3 August 2026

1 Why this is worth building

Risk R2 in the sprint plan says the locked-down sandbox may not serve a port or run a local browser against one. The CLI was the stated fallback — but it loses the two beats that carry the demo.

The CLI cannot show the two things the client asked for

The client's brief named delivery “*through design*”, and the design step is rendered diagrams. The review gate is governance, and a click reads as a decision where typing `y` reads as a prompt. A standalone HTML bundle keeps both, and still needs nothing installed.

Three delivery modes, compared honestly

Property	Server console	Standalone HTML	CLI
Runs with no install	No venv + deps	Yes	No Python
Runs with no open port	No	Yes	Yes
Renders DFD / ER diagrams	Yes	Yes	No
Gate reads as governance	Yes	Cosmetic	No
Gate genuinely enforces	Yes	No see §3	Yes
Emails / shares as one file	No	Yes	No
Produces a run transcript	No	No	Yes

This does not replace the CLI — it sits beside it

The CLI remains the only mode that produces a paste-able transcript and the only one whose output is assertable in `pytest`. The standalone bundle is the *presentation* fallback; the CLI is the *automation* fallback. Sprint 1 still builds the CLI.

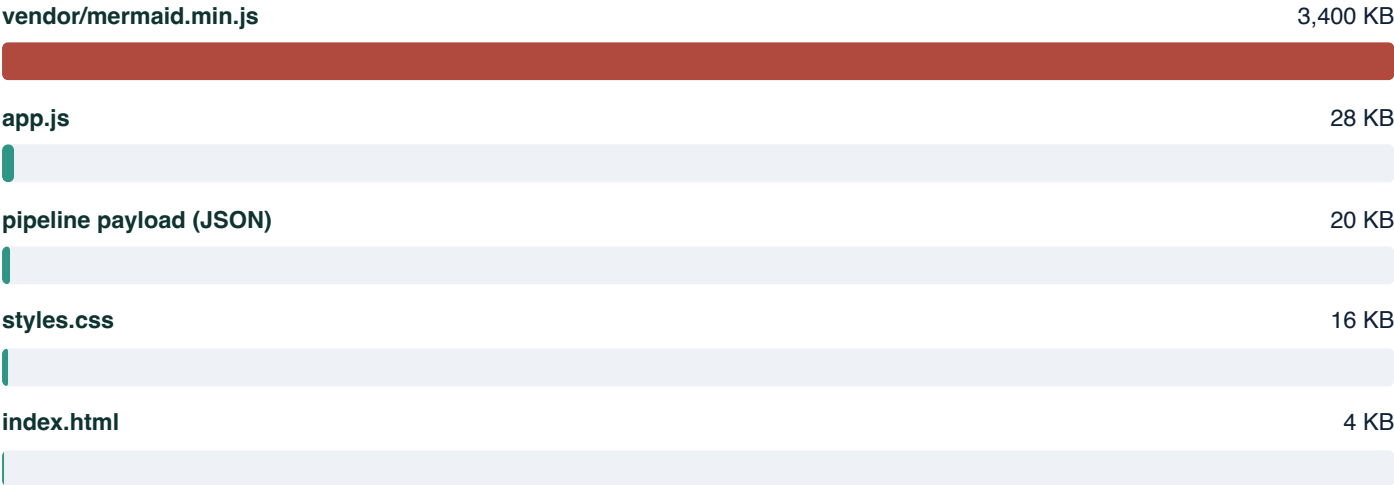
2 What the measurements changed

Two numbers taken from the repo reshaped this plan. Neither was obvious before measuring.

Finding 1 — the coupling is tiny

`apps/console/static/app.js` is 28 KB, and its entire dependence on the backend is **three fetch calls**: `GET /api/run`, `POST /api/gate`, `POST /api/reset`. Everything else is rendering. That is a clean seam, not a rewrite.

Finding 2 — the diagram library is 95% of the weight



The diagrams are static, so the renderer does not need to ship

Mermaid exists to turn diagram *source* into SVG at runtime. But the S7 diagrams never change between runs — they are fixed artifacts. Pre-rendering them to inline SVG at build time removes the 3.4 MB library entirely and takes the bundle from roughly **3.5 MB to about 180 KB**, a ~95% reduction.

The build step needs a browser to run mermaid once — and `demo/render_pdf.py` already drives headless Chrome, so the machinery exists.

Bundle strategy	Size	Trade-off
Inline mermaid as-is	~3.5 MB	Simplest build; too heavy to email; slow first paint
Pre-render diagrams to SVG RECOMMENDED	~180 KB	Build needs headless Chrome once; diagrams become fixed
Link mermaid from a CDN	~150 KB	Rejected — needs network, defeats the purpose

3 The gate problem — decide this first

In a standalone file the gate cannot really enforce anything

Today the gate blocks server-side: the stories **do not exist in the payload** until it is approved, which is why `/api/run` returns `stories: []` and a locked reason. In a single file, all three stories must ship inside the HTML — so the gate can only *hide* them. Anyone who opens developer tools sees them.

This matters more than it looks. `CLAUDE.md` calls the gate “*load-bearing, not optional*”, and the repo's sharpest rule is that a staged artifact presented as a live one is the failure that loses the room. A cosmetic gate presented as an enforcing gate is exactly that failure in a new costume.

The resolution: label it, do not hide it

The standalone build carries a persistent banner, and the gate panel states its own limitation in the same way every artifact already states its provenance:

OFFLINE BUNDLE — NO SERVER

Artifacts are **STAGED**. The review gate is demonstrated client-side; enforcement is server-side in the real pipeline.

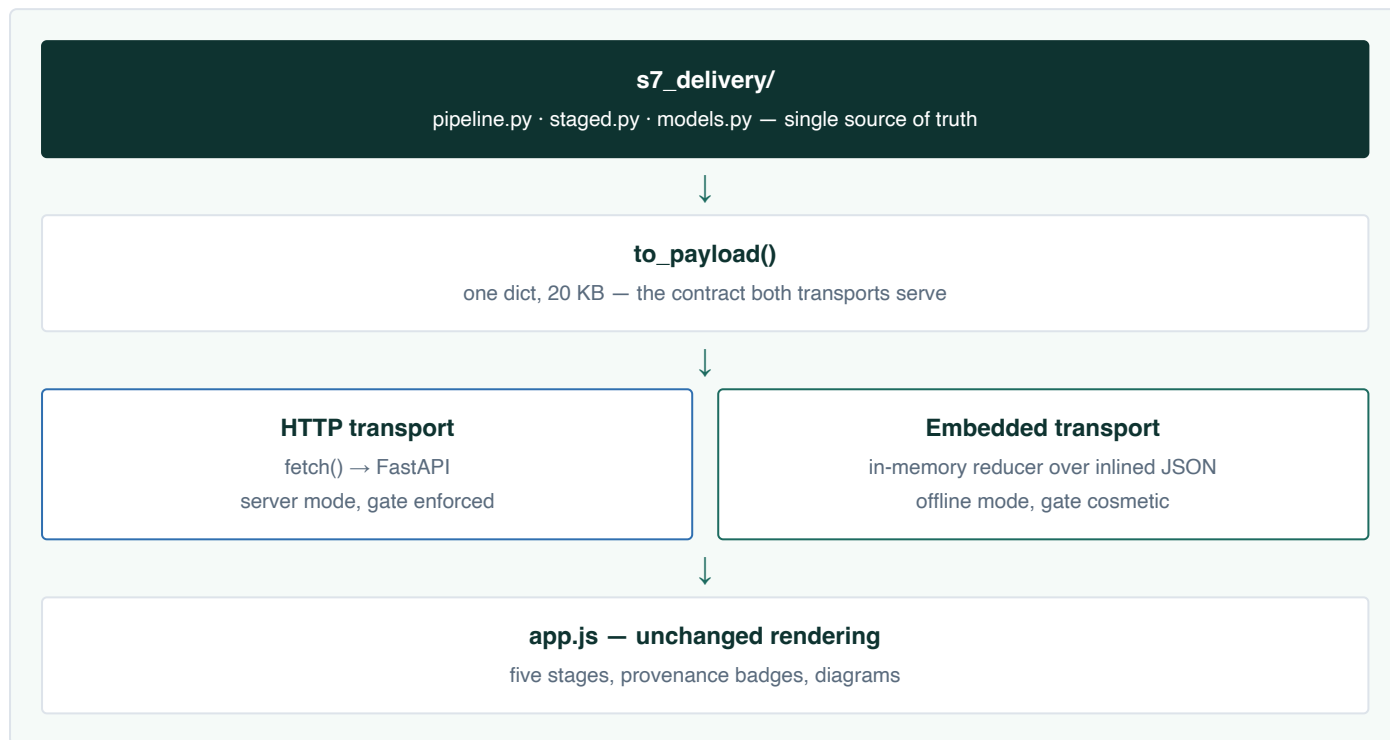
This costs nothing in the room — it is one sentence, and it is the same honesty the demo already trades on. It converts a discoverable weakness into a stated design boundary.

Alternative considered and rejected

The stories could be withheld from the bundle and injected only on approval via an obfuscated blob. That makes the gate *look* enforcing while still being client-side — strictly worse, because it is the same weakness plus an attempt to conceal it. Rejected on the same grounds as any unlabelled staged artifact.

4 Architecture — one pipeline, two transports

The console never learns which mode it is in. A transport object is injected at load; everything above it is unchanged.



The seam, concretely

```
// app.js — today, three hard-coded fetches
fetch("/api/run")           fetch("/api/gate", ...)   fetch("/api/reset", ...)

// after — one injected object, same three verbs
const transport = window.S7_EMBEDDED
  ? embeddedTransport(window.S7_EMBEDDED) // offline bundle
  : httpTransport();                     // served console

transport.run()             transport.gate(d)        transport.reset()
```

The embedded transport is a small reducer: it holds the approved-state payload, and `gate()` returns a filtered copy with `stories` emptied and the locked reason set when the decision is not `approved` — mirroring exactly what `pipeline.build_state()` does server-side.

Why this shape and not a second frontend

A separate offline page would drift from the served console the moment either changed, and the drift would be invisible until a demo. One frontend with two transports cannot drift — and a test asserts the embedded payload matches `to_payload()` byte for byte.

5 Build plan

Four phases. Each leaves the served console working — no phase requires a flag day, and the plan can be stopped after any phase with something of value.

Phase 1 — Extract the transport seam

~½ day · no new files

- Move the three `fetch` calls in `app.js` behind a `httpTransport()` object with `run / gate / reset`.
- Route all call sites through it. No behaviour change.
- Keep the existing offline error path — it is now a transport concern.

EXIT

The served console behaves identically; all 60 tests still green. This phase is pure refactor and is safe to land alone.

Phase 2 — Pre-render the diagrams

~1 day · the interesting one

- Build step loads the console in headless Chrome, waits for mermaid, and extracts each rendered `<svg>`.
- Write them to `s7_delivery/cache/diagrams/*.svg` — committed, like the replay recordings, so a fresh clone needs no browser.
- Served console keeps using mermaid; only the bundle uses the SVGs.

EXIT

Committed SVGs render identically to the live diagrams. Bundle no longer needs the 3.4 MB library.

Phase 3 — The bundler

~1 day

- `demo/build_standalone.py` — imports `s7_delivery`, calls `to_payload()`, inlines CSS, JS, SVGs and the payload into one file.
- Emits `dist/s7-console.html`. Adds the offline banner and the gate labelling from §3.
- Embedded transport implemented as the reducer described in §4.

EXIT

Open the file from `file://` with no server running: five stages render, diagrams draw, the gate toggles stories.

Phase 4 — Lock it against drift

~½ day

- Test: the payload embedded in the bundle equals `to_payload()`.
- Test: the embedded reducer and `pipeline.build_state()` agree on all three gate decisions — **the assertion that stops the two modes diverging**.
- Reuse `inline_images()` to emit the delivery pack as a standalone page alongside the PDF.

EXIT

Both bundles regenerate from one command and cannot silently disagree with the server.

6 What ships, and what could go wrong

Deliverables

Artifact	Size	Purpose
<code>dist/s7-console.html</code>	~180 KB	The demo, offline, one file
<code>dist/s7-delivery-pack.html</code>	~900 KB	The pack as a page rather than a PDF
<code>demo/build_standalone.py</code>	—	Regenerates both
<code>s7_delivery/cache/diagrams/</code>	~40 KB	Committed SVGs; no browser needed to rebuild
<code>tests/test_standalone.py</code>	—	The anti-drift assertions

Risks

#	Risk	Impact	Mitigation
S1	Gate mistaken for enforcing	The credibility failure the repo exists to avoid	Persistent banner + gate-panel label. Non-negotiable, \$3
S2	Bundle drifts from the served console	Two demos that disagree	One frontend, two transports; equality tests in Phase 4
S3	Pre-rendered SVGs go stale	Diagrams show old design	Committed like replay recordings; regenerated by the same build
S4	<code>file://</code> blocks something at demo time	Bundle will not open	No <code>fetch</code> , no modules, no workers — inline everything. Test from <code>file://</code> , not a local server
S5	Scope creep into a second product	Sprint 1 slips	Phases 1–2 are useful alone; stop after any phase

Where this sits against the sprint plan

This is **not** in Sprints 0–5. It is a mitigation for R2 that becomes worth building only if the platform team confirms the sandbox cannot serve a port. Phase 1 is a safe refactor worth doing regardless; Phases 2–4 should wait for that answer. **Ask the platform team before starting Phase 2.**

Open decisions

- **Does the sandbox permit a local port at all?** — the whole trigger. Unanswered.
- **Does the bundle ship the gate at all**, or open post-approval with stories already visible? A labelled cosmetic gate is more faithful to the flow; no gate is more honest still. Recommend the labelled gate.
- **One bundle or two** — console and pack separately, or a single navigable document. Recommend separate; they have different audiences.

Plan only — nothing here is implemented. Sizes are measured from the current repo; the ~180 KB target is an estimate from those measurements, not a built artifact. MapleSure Insurance is a fictional demonstration insurer.